

```
!pip install -q transformers datasets evaluate sentencepiece sacrebleu rouge-score
```

```
Preparing metadata (setup.py) ... done
84.1/84.1 kB 4.6 MB/s eta 0:00:00
100.8/100.8 kB 7.6 MB/s eta 0:00:00
Building wheel for rouge-score (setup.py) ... done
```

```
import torch
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from datasets import Dataset
from evaluate import load
from sklearn.model_selection import train_test_split
from google.colab import files
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
uploaded = files.upload()

csv_file = list(uploaded.keys())[0]
print("Uploaded file:", csv_file)

df = pd.read_csv(csv_file)

print("\nDataset Preview:")
print(df.head())

print("\nColumns:", df.columns)
```

Choose Files | faq.csv
faq.csv(text/csv) - 846 bytes, last modified: 3/24/2026 - 100% done
 Saving faq.csv to faq.csv
 Uploaded file: faq.csv

Dataset Preview:

```

      question \
0    What is a savings account?
1  How can I open a bank account?
2           What is KYC?
3    Can I transfer money online?
4           What is a debit card?

      answer
0  A savings account is a deposit account that ea...
1  You can open an account by visiting a branch o...
2  KYC means Know Your Customer and is required f...
3  Yes you can transfer money using net banking o...
4  A debit card allows you to withdraw money from...
```

```
Columns: Index(['question', 'answer'], dtype='object')
```

```
train_q, test_q, train_a, test_a = train_test_split(
    df["question"].tolist(),
    df["answer"].tolist(),
    test_size=0.3,
    random_state=42
)
```

```
train_dataset = Dataset.from_dict({
    "question": train_q,
    "answer": train_a
})
```

```
test_dataset = Dataset.from_dict({
    "question": test_q,
    "answer": test_a
})
```

```
model_name = "google/mt5-small"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%                               553/553 [00:00<00:00, 59.6kB/s]

tokenizer_config.json: 100%                       82.0/82.0 [00:00<00:00, 8.89kB/s]

spiece.model: 100%                               4.31M/4.31M [00:00<00:00, 62.1MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
special_tokens_map.json: 100%                     99.0/99.0 [00:00<00:00, 6.42kB/s]

pytorch_model.bin: 100%                           1.20G/1.20G [00:08<00:00, 124MB/s]

model.safetensors: 42%                           503M/1.20G [00:05<00:07, 98.3MB/s]

Loading weights: 100%                             192/192 [00:00<00:00, 211.37it/s, Materializing param=shared.weight]

The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but both are p
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but both are p
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the
generation_config.json: 100%                       147/147 [00:00<00:00, 3.83kB/s]
```

```
max_len = 128

def preprocess(batch):
    inputs = tokenizer(
        batch["question"],
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    targets = tokenizer(
        batch["answer"],
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    labels = [
        [(t if t != tokenizer.pad_token_id else -100) for t in seq]
        for seq in targets["input_ids"]
    ]

    inputs["labels"] = labels
    return inputs

train_tokenized = train_dataset.map(preprocess, batched=True)
test_tokenized = test_dataset.map(preprocess, batched=True)

train_tokenized.set_format("torch")
test_tokenized.set_format("torch")
```

```
Map: 100%                               7/7 [00:00<00:00, 195.33 examples/s]

Map: 100%                               3/3 [00:00<00:00, 129.36 examples/s]
```

```
from transformers import Trainer, TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=3e-5,
    per_device_train_batch_size=2,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_steps=10,
    save_steps=500,
    report_to=[]
)
```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized
)

print("\n=== TRAINING STARTED ===")
trainer.train()

```

=== TRAINING STARTED ===

[12/12 01:07, Epoch 3/3]

Step Training Loss

Step	Training Loss
10	26.517926

Writing model shards: 100%

1/1 [00:30<00:00, 30.70s/it]

TrainOutput(global_step=12, training_loss=28.082550048828125, metrics={'train_runtime': 68.8614, 'train_samples_per_second': 0.305, 'train_steps_per_second': 0.174, 'total_flos': 2775935877120.0, 'train_loss': 28.082550048828125, 'epoch': 3.0})

```

model.eval()

predictions = []
references = []

for i in range(len(test_dataset)):
    input_text = test_dataset[i]["question"]

    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        truncation=True,
        padding=True
    ).to(device)

    outputs = model.generate(
        **inputs,
        max_new_tokens=64,
        num_beams=4
    )

    pred = tokenizer.decode(outputs[0], skip_special_tokens=True)
    ref = test_dataset[i]["answer"]

    predictions.append(pred)
    references.append(ref)

```

```

from evaluate import load

rouge = load("rouge")
bleu = load("bleu")

rouge_results = rouge.compute(
    predictions=predictions,
    references=references
)

bleu_results = bleu.compute(
    predictions=predictions,
    references=references
)

print("\n=== EVALUATION RESULTS ===")
print("ROUGE-L Score:", round(rouge_results["rougeL"], 4))
print("BLEU Score   :", round(bleu_results["bleu"], 4))

```

=== EVALUATION RESULTS ===
 ROUGE-L Score: 0.0
 BLEU Score : 0.0

```

print("\n=== SAMPLE PREDICTIONS ===")

for i in range(min(3, len(predictions))):
    print("Q :", test_dataset[i]["question"])

```

```
print("Pred:", predictions[i])
print("Ref :", test_dataset[i]["answer"])
print("-"*60)
```

```
=== SAMPLE PREDICTIONS ===
Q : How to check my account balance?
Pred: <extra_id_0>
Ref : You can check via ATM SMS or mobile banking.
-----
Q : How can I open a bank account?
Pred: <extra_id_0>
Ref : You can open an account by visiting a branch or applying online.
-----
Q : How to reset my ATM PIN?
Pred: <extra_id_0>
Ref : You can reset it using ATM machine or internet banking.
-----
```

Start coding or [generate](#) with AI.